

Poly Language Specification

A Simple, Assembly-Inspired System Programming Language Transpiling to Rust

Version: 1.3 Draft

Target Backend: Rust (Cargo Workspace)

Status: Active Definition

1. Introduction & Core Philosophy

Poly is designed as a minimalist, low-overhead system programming language with a clean, highly explicit syntax reminiscent of assembly language. It provides fine-grained control over low-level memory and ASCII/Unicode data primitives while transpiling directly into safe, idiomatic Rust. By outputting Rust source code inside a standard Cargo project structure, Poly leverages Rust's memory safety guarantees, optimized LLVM compilation, and rich crate ecosystem without exposing developers to complex syntax overhead.

2. Variables, Constants & Assignment

Poly distinguishes between initial declarations and subsequent variable updates to maintain strict type safety while reducing syntactic verbosity.

- `var` is **required** during variable declaration with an explicit type annotation.
- `var` is **optional** for subsequent reassignments to already-declared variables.
- `const` defines immutable, compile-time constants.

Code Example

```
// Constant definition
const MAX_BUFFER = 1024

// Variable declaration (requires 'var' and explicit type)
var count: i32 = 0
var status: bool = true

// Reassignment ('var' is optional once declared)
count = 5
count = count * 2
```

3. Arithmetic Operations

Poly supports both explicit assembly-like arithmetic instructions (`add` , `sub`) and high-level compound assignment operators (`+=` , `-=`). When using assembly-style instructions, omitting the step value automatically defaults the increment or decrement to `1`.

Poly Syntax	Default Step	Transpiled Rust Output
<code>add counter</code>	1	<code>counter += 1;</code>
<code>add counter, 10</code>	Explicit (10)	<code>counter += 10;</code>
<code>sub score</code>	1	<code>score -= 1;</code>
<code>sub score, 15</code>	Explicit (15)	<code>score -= 15;</code>
<code>counter += 5</code>	N/A	<code>counter += 5;</code>

Code Example

```
var index: i32 = 0
var health: i32 = 100

// Assembly-style increment / decrement
add index          // index becomes 1
add index, 5        // index becomes 6
sub health          // health becomes 99
sub health, 20      // health becomes 79

// Equivalent operator syntax
index += 1
health -= 5
```

4. Data Types: Characters & Strings

To facilitate precise system level programming, Poly separates raw byte/ASCII data from variable-length Unicode data at the type level.

Poly Type	Description	Memory Size	Transpiled Rust Mapping
<code>char</code>	Single ASCII character	1 byte (u8)	<code>u8</code>
<code>string</code>	Sequence of ASCII characters / bytes	1 byte / char	<code>Vec<u8></code> / <code>&[u8]</code>
<code>uchar</code>	Single UTF-32 / Unicode scalar value	4 bytes	<code>char</code>
<code>ustring</code>	UTF-8 encoded Unicode text string	1–4 bytes / char	<code>String</code> / <code>&str</code>

Code Example

```
// ASCII Character and String Literals
var letter: char = 'A'
var header: string = "HTTP/1.1 200 OK"

// Unicode Character and String Literals
var symbol: uchar = u'🚀'
var message: ustring = u"Hello, 世界! 🚀"
```

5. Structs & Custom Types

Struct fields can be defined either with explicit `var` field descriptors or simple name-type pairs. Every struct block must be explicitly closed with `end struct`.

Code Example

```
struct Point
  var x: f32
  y: f32
end struct

struct PacketHeader
  id: i32
  length: i32
  signature: char
end struct
```

6. Direct Memory Operations & Raw Pointers

Poly allows unconstrained raw pointer access for direct hardware manipulation and interop. Raw pointers are designated via `ptr T`, and dereferencing is performed explicitly using the `deref` keyword. During compilation, raw pointer manipulations are automatically wrapped in Rust `unsafe` blocks.

Code Example

```
fn inspect_memory(address: ptr i32): i32
  if address != 0 then
    // Explicit instruction-style dereference
    var value: i32 = deref address
    return value
  else
    return 0
  end if
end fn
```

7. Functions & Error Handling

Functions begin with the `fn` keyword and terminate with `end fn`. Error handling uses Rust's explicit result semantics via the `try` keyword (replacing Rust's `?` operator).

Code Example

```
fn read_file_size(path: ustring): Result<i32, Error>
  // 'try' handles error propagation explicitly
  var file = try open_file(path)

  if file.is_valid then
    return Ok(file.size)
  else
    return Err(Error.InvalidFile)
  end if
end fn
```

8. Control Flow & Loops

Control flow constructs feature strict block matching markers:

- `if ... then ... else ... end if`

- `while ... end while`
- `loop ... end loop`

Code Example

```
fn process_buffer(buffer_ptr: ptr i32, limit: i32)
  var i: i32 = 0

  while i < limit
    if buffer_ptr != 0 then
      add i
    else
      add i, 2
    end if
  end while
end fn
```

9. Transpilation & Target Project Structure

The Poly transpiler outputs a complete standard Cargo package structure, allowing seamless integration with Cargo dependencies and tools:

```
my_poly_project/
├─ Cargo.toml
├─ src/
│  └─ main.rs
```

Summary of Poly to Rust Transpilation Mapping

Poly Language Construct	Transpiled Rust Output
<code>var x: i32 = 10</code>	<code>let mut x: i32 = 10;</code>
<code>add x, 5 / x += 5</code>	<code>x += 5;</code>
<code>ptr i32</code>	<code>*mut i32 / *const i32</code>
<code>var v = deref p</code>	<code>let mut v = unsafe { *p };</code>
<code>var f = try open()</code>	<code>let mut f = open()?;</code>
<code>end if / end fn / end while</code>	<code>}</code>